CHATBOT POUR LA LOI DE TRAVAIL MAROCAINE

Retreival Augmented Generation

Réalisé par :
Rajaa LEBNAITI

Table des matières

1) Préparation des données et chunking 5

1-1) Chunking 5

- Regex 5

2) Choix de la base de données et indexation 6

3) Retrieval hybrid and re-ranking 6

1-1) Hybrid Search 6

2-1) Fusionner les résultats (RRF) 7

3-1) Reranking 7

4) LLM, Prompt engineering et paramètres 8

1-1) Architecture 8

2-1) Prompt Engineering & System Prompt 8

- System Prompt 8
- In context learning and chain of thought 9

5) Évaluation, Contrôle des Hallucinations & Observabilité 9

- Gestion des hallucinations ..... **Erreur ! Signet non défini.**
- Evaluation de la qualité avec RAGAS 9
- Observabilité avec Phoenix de Arize 9

Liste des tableaux

Tableau 1: Les paramètres du modèle 8

Liste des figures

Figure 1: Code 1 Chunking using Regex 5

Figure 2: Architecture globale du projet AI-Generated Erreur ! Signet non défini.

Plan de travail

1) Préparation des données et chunking

1-1) Chunking

Il existe plusieurs méthodes pour faire le découpage des documents. Dans ce projet, on va utiliser la méthode de Regex.

Regex

Cette méthode permet de découper un document textuel en des segments (chunks) selon des règles linguistiques sur texte (phrases, paragraphes, mots) plutôt que par un simple nombre de caractères ou de mots.

Les morceaux obtenus contiennent des idées complètes et linguistiquement cohérentes, ce qui permet de réduire l'hallucination du modèle.

Pour cela on va utiliser la formule regex suivante :

```python
pattern = r"(\nArticle\s*(?:premier|\d+))"
parts = re.split(pattern, full_text)

chunks = []

for i in range(1, len(parts), 2):
    title = parts[i].strip()
    content = parts[i+1].strip()



    full_article_text = f"{title}\n{content}"
    clean_title = re.sub(r"\s+", " ", title)


    chunks.append(Document(
        page_content=full_article_text,
        metadata={"article_no": clean_title, "source": "Code du Travail Marocain"}
    ))
```

Figure 1: Code 1 Chunking using Regex

On obtient : **1chunk = 1 document = 1 article.**

On a aussi attaché à chaque article des métadonnées :

{ "article_no": "Article 16",

"source": "Code du travail marocain" }

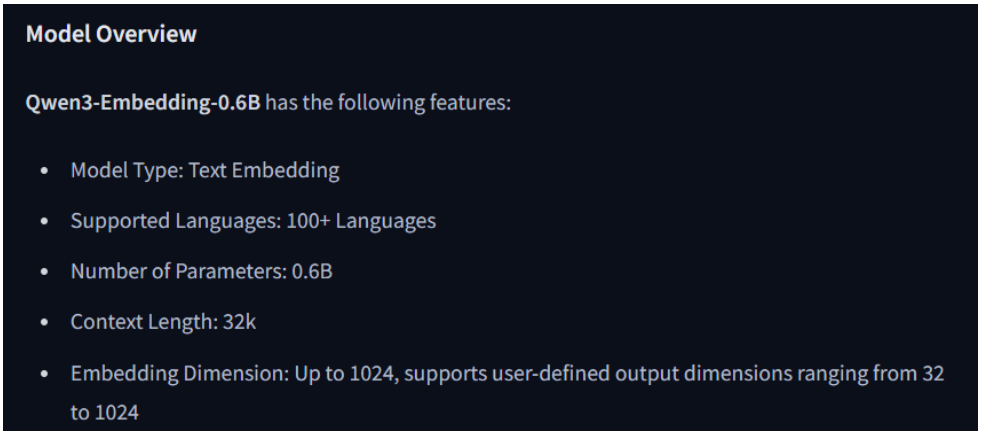
2) Choix de la base de données et indexation

Il existe plusieurs types de base de données vectorielle comme ; ChromaDb, FAISS et Weaviate.

Pour ce projet on va utiliser ChromaDb qui permet de gérer à la fois les vecteurs et le filtrage par métadonnées. Elle s'installe en local par *pip install chromadb*

Pour la partie embedding, on va utiliser Qwen3-Embedding :0.6B

(Hugging Face, s.d.)



Ce modèle est relativement léger, il supporte plusieurs langues, il a une bonne capacité d'embedding et il est adapté à nos contraintes matérielles donc c'est un bon compromis qualité/ ressources/ coût.

3) Retrieval hybrid and re-ranking

1-1) Hybrid Search

Dans ce projet, on va adopter une approche hybride de recherche.

On va utiliser à la fois la recherche par mots (***keyword search avec BM25***) et aussi recherche par sens (***semantic search avec chroma_retriever***).

```python
from langchain_classic.retrievers import EnsembleRetriever

bm25_retriever = BM25Retriever.from_documents(documents)
bm25_retriever.k = 10

chroma_retriever = vector_store.as_retriever(search_kwargs={"k": 10})

# Recherche hybride
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, chroma_retriever],
    weights=[0.5, 0.5]
)
print(type(hybrid_retriever))
```

Figure 2: Hybrid search with RRF

2-1) Fusionner les résultats (RRF)

Vu qu'on va travailler avec une approche hybride, alors le BM25 va donner une liste de documents pertinents ainsi que la recherche sémantique va aussi donner une autre liste.

C'est pourquoi la méthode RRF (***Reciprocal Rank Fusion***) « déjà implémentée dans *EnsembleRetriever* » permet de fusionner ces deux listes en un score unifié. Cela garantit que si un article contient le mot-clé ET est sémantiquement proche, il remonte en haut.

3-1) Reranking

On va utiliser un Cross-Encoder nommé **FlashrankRerank** (modèle open-source léger) pour évaluer la liste des documents récupérer, les classer en plaçant les **5 articles les plus pertinents** de liste résultante du RRF au début avant de l'envoyer au LLM.

```python
from langchain_classic.retrievers.contextual_compression import ContextualCompressionRetriever
from langchain_community.document_compressors import FlashrankRerank

compressor = FlashrankRerank(model="ms-marco-MiniLM-L-12-v2",top_n = 5) # on garde juste top 5


final_retriever = ContextualCompressionRetriever(
    base_compressor = compressor,
    base_retriever = hybrid_retriever
```

Figure 3: Reranking

4) LLM, Prompt engineering et paramètres

1-1) Architecture

L'architecture qu'on va adopter est un simple RAG (sans agentic rag).

C'est un pipeline linéaire où on a :

Requête → Retreival → Generation

Pour le modèle LLM on va utiliser le modèle local via Ollama : **Qwen 2.5 (7B / 14B)**

Le modèle est 100% gratuit, il est fort en logique et francophonie, il a 128k tokens ce qui est largement suffisant pour notre cas d'utilisation.

Les paramètres utilisés pour ce modèle :

Pour un chatbot juridique, on a besoin d'un comportement qui soit déterministe (pas de créativité). Pour ce faire, on va adopter les paramètres suivants :

Paramètre	Valeur	Justification
Température	0.0 ou 0.1 (proche de 0)	Le modèle accentue les mots à fort probabilités (pas de créativité)
Top P	0.9	Permet de garder une structure grammaticale fluide
Repetition penalty	1.1 à 1.2	Évite que le modèle ne boucle sur les formulations juridiques répétitives.

Tableau 1: Les paramètres du modèle

2-1) Prompt Engineering & System Prompt

System Prompt

On va utiliser le system prompt pour ajouter des contraintes comme :

- Le modèle ne doit répondre que en se basant sur les documents fournis.

- Si la réponse n’est pas dans les documents, le modèle dis poliment qu’il ne sait pas.
- Le modèle doit citer ses sources.

In context learning and chain of thought

Dans la prompt, on va donner des exemples en utilisant la méthode « ***few shot*** » cela permet au modèle de savoir comment décomposer un problème.

5) Évaluation, Contrôle des Hallucinations & Observabilité

Après avoir implémenter tout cela, on doit évaluer le projet.

Evaluation de la qualité avec RAGAS

On va préparer un petit Dataset de test (10-215 questions clés avec les réponses attendues et les vrais articles associés).

Puis on va utiliser RAGAS pour évaluer le RAG sur 3 piliers :

- **Faithfulness** : Est-ce que la réponse générée est entièrement basée sur le context.
- **Response Relevance :** Est-ce que la réponse répond vraiment à la question de l'employé.
- **Context Recall :** Est-ce que le moteur de recherche a bien réussi à retrouver les bons articles de loi.

Observabilité avec Phoenix de Arize

On va utiliser Phoenix, c’est un outil open-source pour visualiser tes traces de RAG. On peut voir exactement ce que l'utilisateur a tapé, quel chunk a été récupéré par ChromaDB, comment le prompt a été formaté, et ce que le LLM a répondu.